Lecture 4: Graph-based spectral methods, kernel PCA for representation learning

TTIC 31220: Unsupervised Learning and Data Analysis

Instructor: Karen Livescu

TTI-Chicago

April 1, 2026

Announcements

- Corrected Lecture 3 slides posted
- Tutorial 2 slides are on canvas
- Final exam schedule has been announced: Our poster session will be Wed. 5/27 1-3pm

Plan for today

Nonlinear dimensionality reduction

- Graph-based spectral methods
- Kernel PCA

Nonlinear dimensionality reduction

What if our data lies on a nonlinear manifold?[1]





(color represents value in low-dimensional space)

We will start by reviewing MDS, which we'll extend to develop nonlinear methods...

[1] a topological space locally "resembling" Euclidean space

Multidimensional scaling

- Sometimes it is natural to consider the input to be the *distances* or *similarities* between data points: document co-occurrence counts, web page link matrices, network ping times, ...

Multidimensional scaling

- Sometimes it is natural to consider the input to be the *distances* or *similarities* between data points: document co-occurrence counts, web page link matrices, network ping times, ...
- MDS seeks a k-dimensional representation $\mathbf{z}$ of a data set that preserves inner products between pairs of data points $\mathbf{x}$:

$$\min \sum_{ij} (\mathbf{x}_i \cdot \mathbf{x}_j - \mathbf{z}_i \cdot \mathbf{z}_j)^2$$

Multidimensional scaling

- Sometimes it is natural to consider the input to be the *distances* or *similarities* between data points: document co-occurrence counts, web page link matrices, network ping times, ...
- MDS seeks a k-dimensional representation $\mathbf{z}$ of a data set that preserves inner products between pairs of data points $\mathbf{x}$:

$$\min \sum_{ij} (\mathbf{x}_i \cdot \mathbf{x}_j - \mathbf{z}_i \cdot \mathbf{z}_j)^2$$

- The solution can be obtained from the $N \times N$ Gram matrix of inner products $\mathbf{G} = \mathbf{X}^T\mathbf{X}$:

$$\mathbf{G}_{ij} = \mathbf{x}_i \cdot \mathbf{x}_j$$

- The output projections are given by

$$\mathbf{z}_{i\alpha} = \sqrt{\lambda_\alpha}\mathbf{v}_{\alpha i},\ 1 \leq \alpha \leq k$$

where $\mathbf{v}_\alpha$ is the αth e-vector of $\mathbf{G}$ and λ_α the corresponding e-value.

Multidimensional scaling

Special case: Start with the matrix of Euclidean distances

$$\mathbf{F}_{ij} = ||\mathbf{x}_i - \mathbf{x}_j||^2 = ||\mathbf{x}_i||^2 - 2\mathbf{x}_i^T\mathbf{x}_j + ||\mathbf{x}_j||^2$$

If the $\mathbf{x}_i$ are zero-mean, we can convert the Euclidean distance matrix to a Gram matrix (inner-product matrix) by left- and right-multiplying by the *centering matrix*:

$$\mathbf{G} = -\frac{1}{2}(\mathbf{I} - \mathbf{u}\mathbf{u}^T)\mathbf{F}(\mathbf{I} - \mathbf{u}\mathbf{u}^T)$$

where $\mathbf{u} = \frac{1}{\sqrt{N}}(1, 1, \ldots, 1)^T$

Multidimensional scaling

Special case: Start with the matrix of Euclidean distances

$$\mathbf{F}_{ij} = ||\mathbf{x}_i - \mathbf{x}_j||^2 = ||\mathbf{x}_i||^2 - 2\mathbf{x}_i^T\mathbf{x}_j + ||\mathbf{x}_j||^2$$

If the $\mathbf{x}_i$ are zero-mean, we can convert the Euclidean distance matrix to a Gram matrix (inner-product matrix) by left- and right-multiplying by the *centering matrix*:

$$\mathbf{G} = -\frac{1}{2}(\mathbf{I} - \mathbf{u}\mathbf{u}^T)\mathbf{F}(\mathbf{I} - \mathbf{u}\mathbf{u}^T)$$

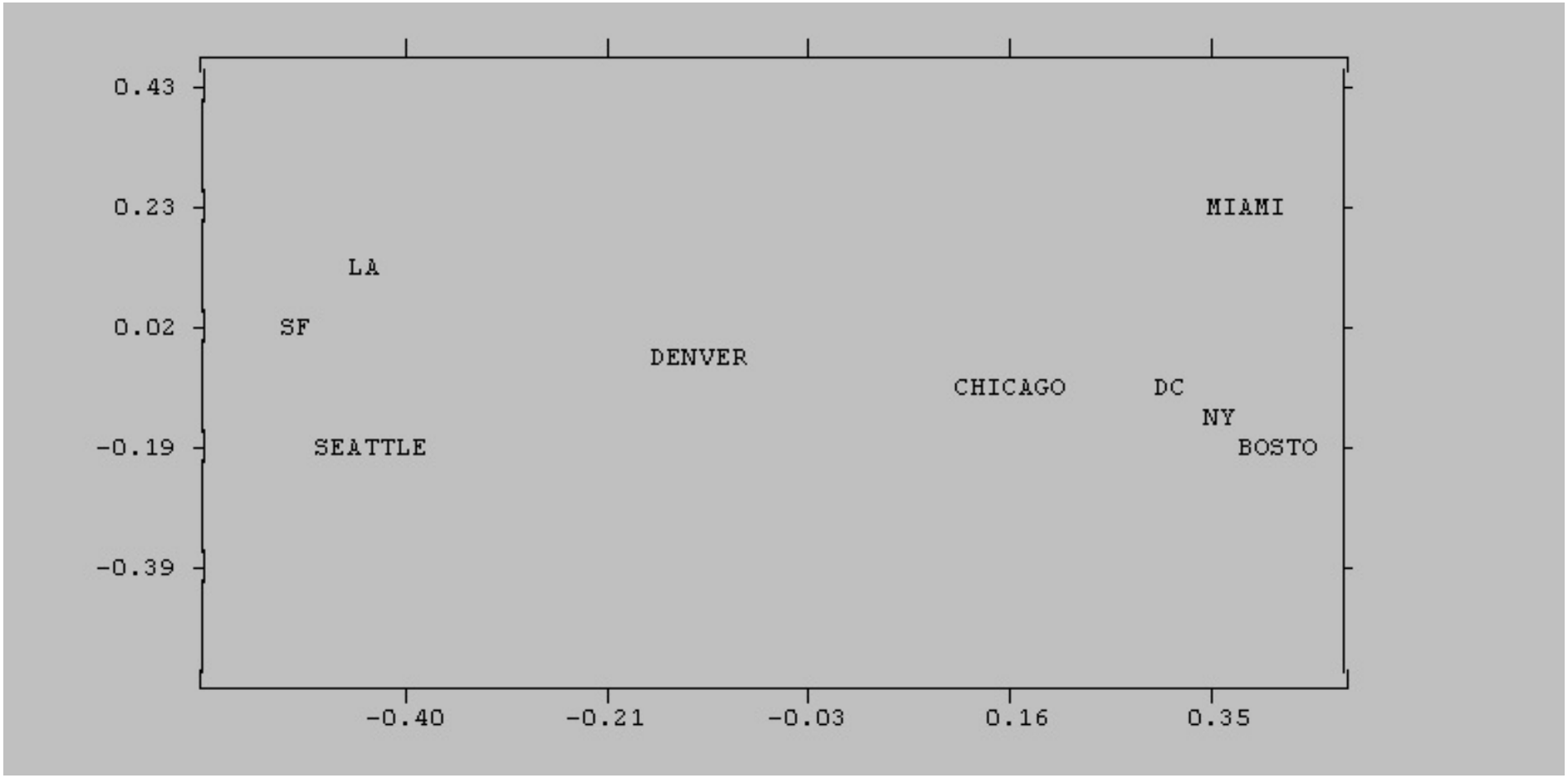
where $\mathbf{u} = \frac{1}{\sqrt{N}}(1, 1, \ldots, 1)^T$

In this case, the MDS projections $\mathbf{z}_i$ are the same as those of PCA:

- PCA directions are eigenvectors of $\mathbf{S} = \mathbf{X}\mathbf{X}^T$
- For each e-vector $\mathbf{w}_l$ of $\mathbf{S}$, there is a corresponding e-vector $\mathbf{v}_l = \mathbf{X}^T\mathbf{w}_l$ of $\mathbf{G} = \mathbf{X}^T\mathbf{X}$
- The first k vectors $\mathbf{v}_l$ give the projected data in both PCA and MDS

MDS example

Input: inter-city distances



Multidimensional scaling

Note:

- Unlike PCA, MDS only gives projections for the training set; it does not give us a way to project a new data point
- We will need an "out-of-sample extension" to project new data points
- We'll come back to this later

Graph-based spectral methods

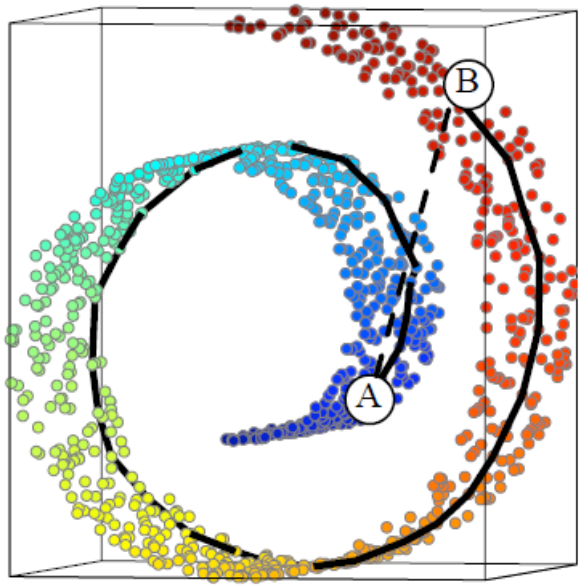
Graph-based spectral methods take inspiration from MDS, extended to a variety of distance/similarity measures

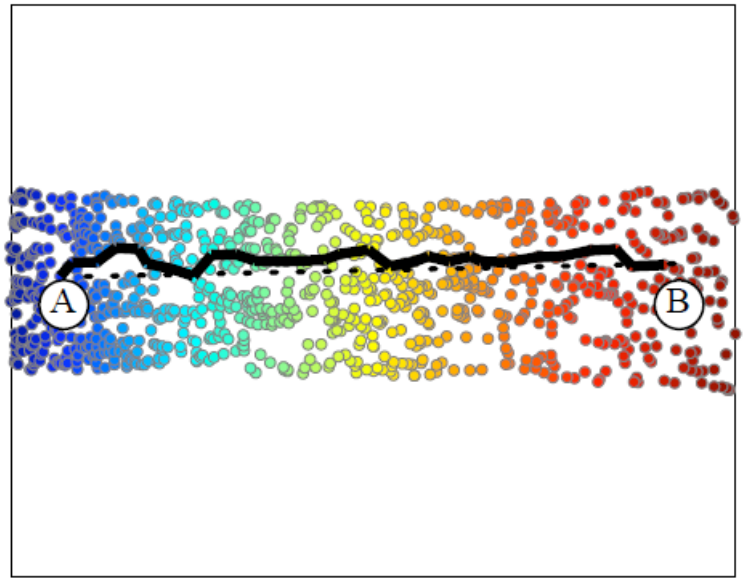
- Construct a graph where each node corresponds to a data point and edges correspond to neighbor relationships
- Construct a matrix summarizing the graph
- Compute spectral decomposition of the matrix, and use it to define the data projections

Examples: Isomap, maximum variance unfolding, locally linear embedding, Laplacian eigenmaps

Isomap [Tenenbaum et al. 2000]

A direct extension of MDS where Euclidean distances are replaced with estimated geodesic distance along the manifold





Isomap [Tenenbaum et al. 2000]

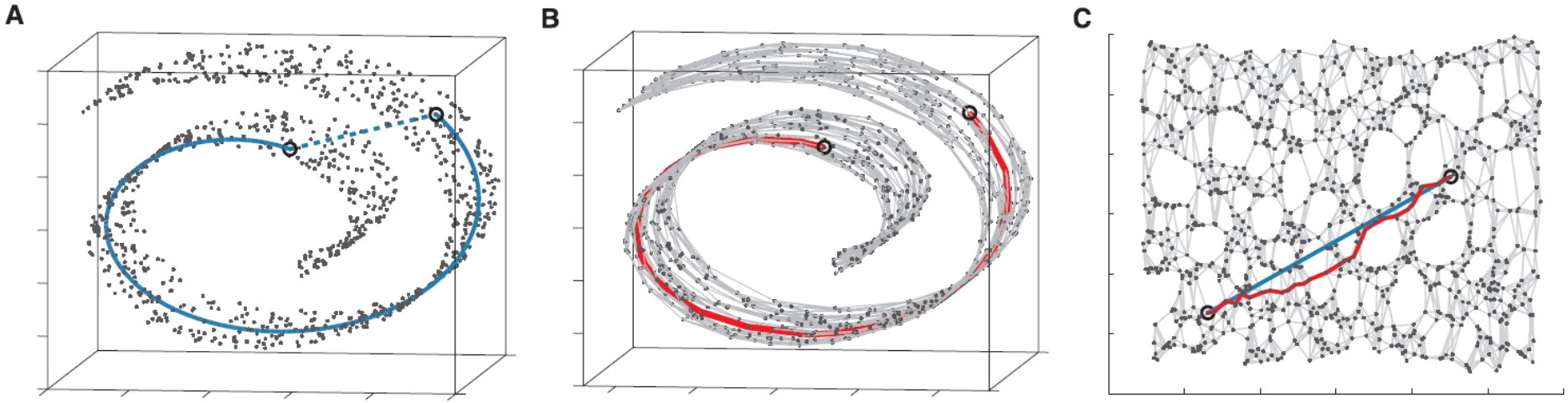
Algorithm:

1. Construct a graph where node i corresponds to input example $\mathbf{x}_i$, and nodes i and j are connected by an edge if $\mathbf{x}_i, \mathbf{x}_j$ are K-nearest (Euclidean) neighbors
2. For each i, j compute pairwise distance Δ_{ij} along the shortest path in the graph (e.g., using Dijkstra's shortest path algorithm)
3. Perform MDS for some dimensionality k, taking the distances Δ_{ij} as input in place of Euclidean distances

The output is the k-dimensional projections of all of the data points $\mathbf{z}_i$. These satisfy $||\mathbf{z}_i - \mathbf{z}_j||^2 \approx \Delta_{ij}^2$

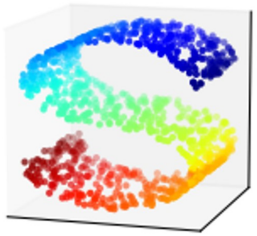
Isomap [Tenenbaum et al. 2000]

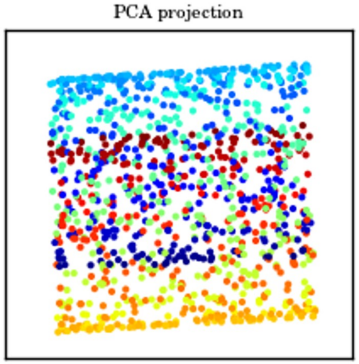
Example with $d = 3, k = 2$

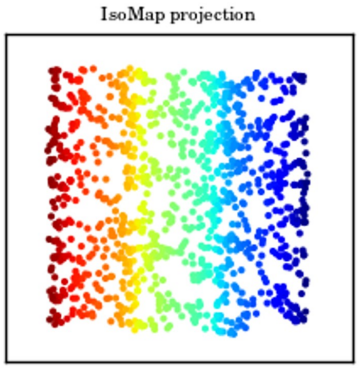


Isomap success

- As the data set size increases, Isomap is guaranteed to converge to the correct manifold that the data was drawn from
- As long as the data set has no "holes" (if it does then geodesic distance computation can break)

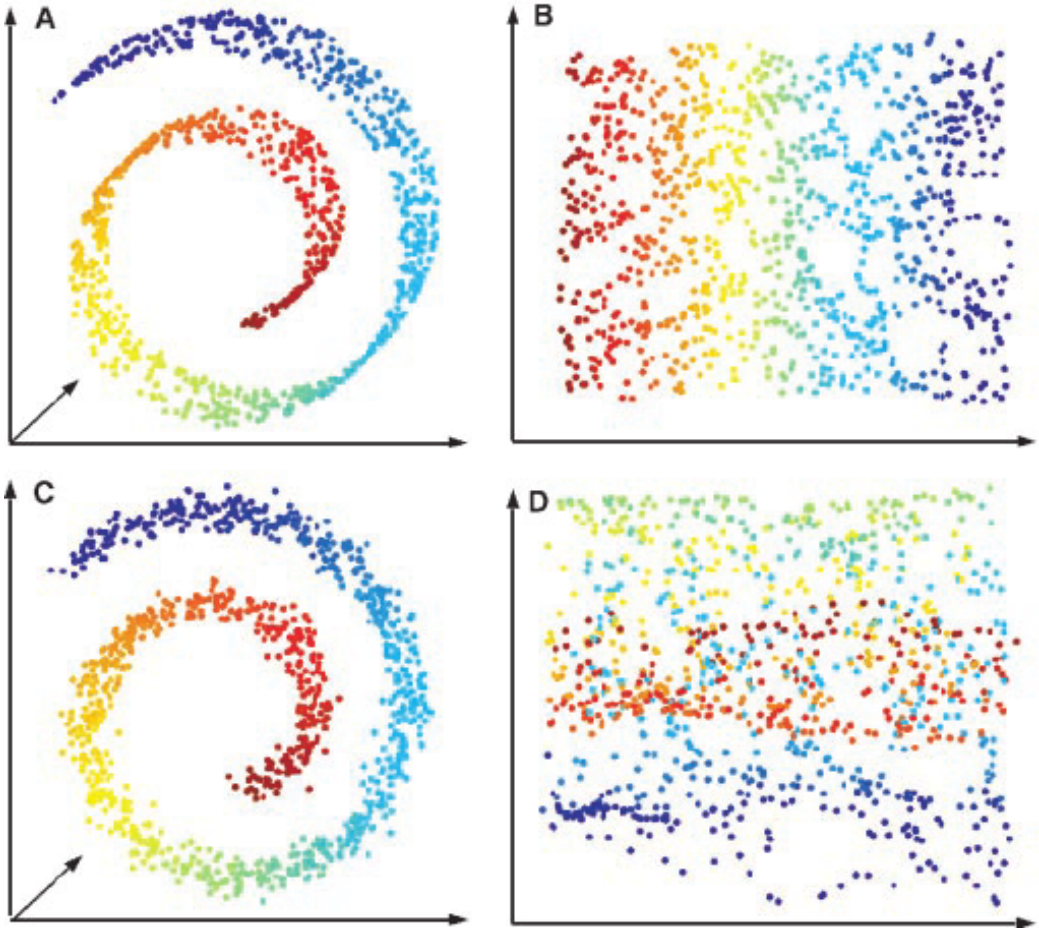






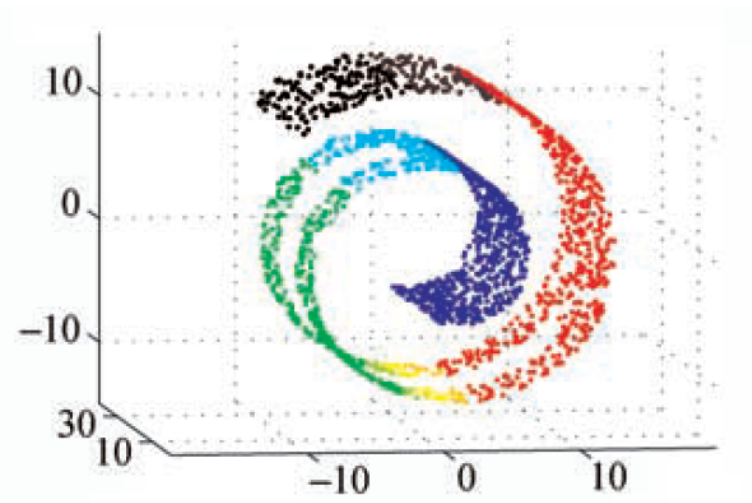
Isomap failure

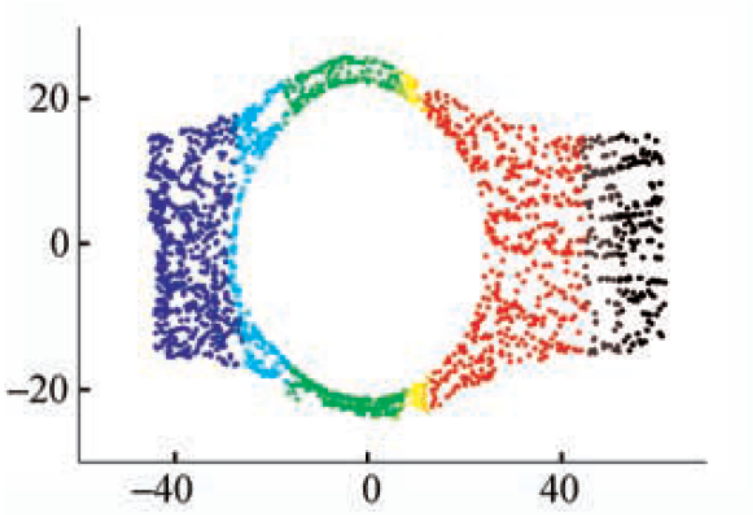
Can be sensitive to the neighborhood size used in graph construction, or equivalently to the noise in the data



Isomap failure

Can't handle holes in the manifold





Laplacian eigenmaps [Belkin & Niyogi 2003]

1. Construct neighborhood graph: node i corresponds to input xi
2. Assign weights to the graph edges, typically

$$W_{ij} = \begin{cases} \exp(-||\mathbf{x}_i - \mathbf{x}_j||^2/t), & ||\mathbf{x}_i - \mathbf{x}_j|| < \epsilon \\ 0 & \text{otherwise} \end{cases}$$

   where t is a tuning parameter
3. Define a diagonal matrix D with $D_{ii} = \sum_j W_{ij}$
4. Find the projections $\mathbf{z}_i$ that solve

$$\min \sum_{ij} \frac{W_{ij}||\mathbf{z}_i - \mathbf{z}_j||^2}{\sqrt{D_{ii}D_{jj}}}$$

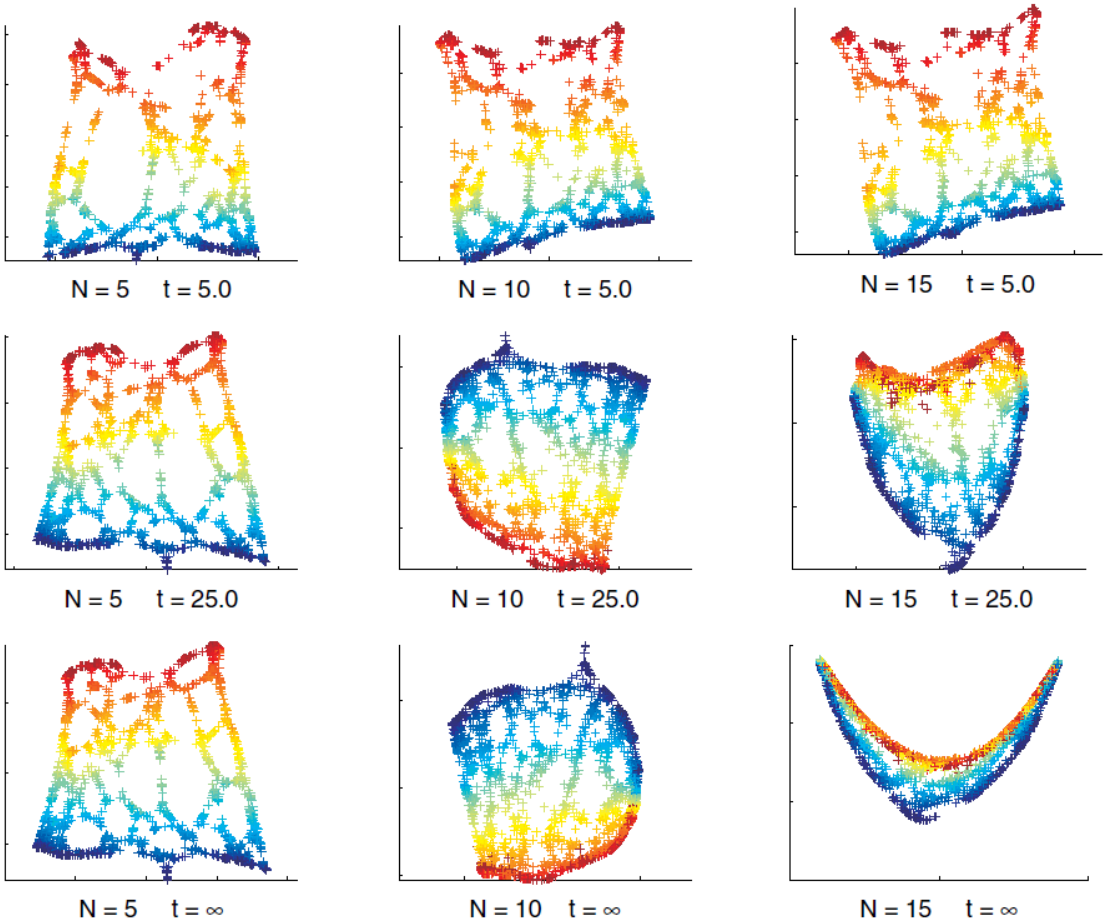
   s.t. the projections are centered and have unit covariance.
5. The solution is given by the k *bottom* eigenvectors of

$$\mathcal{L} = I - D^{-\frac{1}{2}} W D^{-\frac{1}{2}}$$

   excluding the bottom (constant) eigenvector. This is a symmetrized, normalized form of the *graph Laplacian* $D - W$

Laplacian eigenmaps swiss roll examples

Tends to try to cluster the data



(Note: In this version of LE, edge weights were also set to zero for all nodes that are not top-K nearest neighbors of each other.)

Others...

- **Locally linear embedding**: Learns a mapping in which each point can be expressed as a linear function of its nearest neighbors
- **Maximum variance unfolding**: Tries to maximize the variance of the data while respecting neighborhood relationships
- And more...

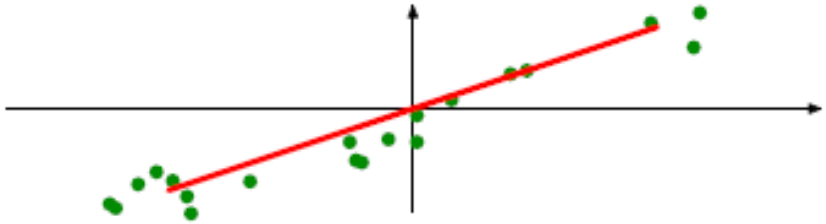
Kernel-based dimensionality reduction

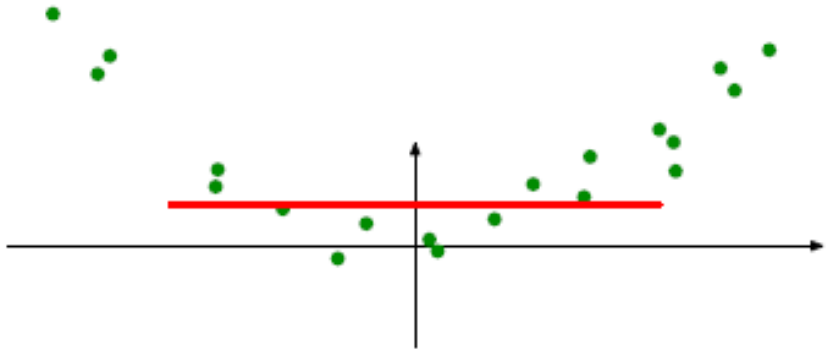
Kernel PCA

A nonlinear extension of PCA.

But first, let's think what problems we expect this to solve for us...

PCA success vs. failure



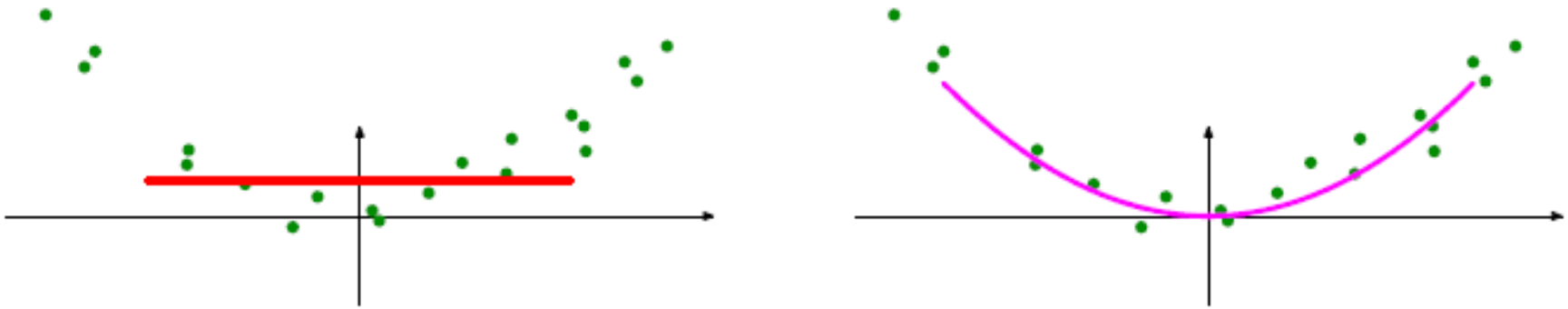


The problem on the right is that we only consider linear subspaces:

$$\hat{\mathbf{x}} = \mathbf{W}\mathbf{z}$$

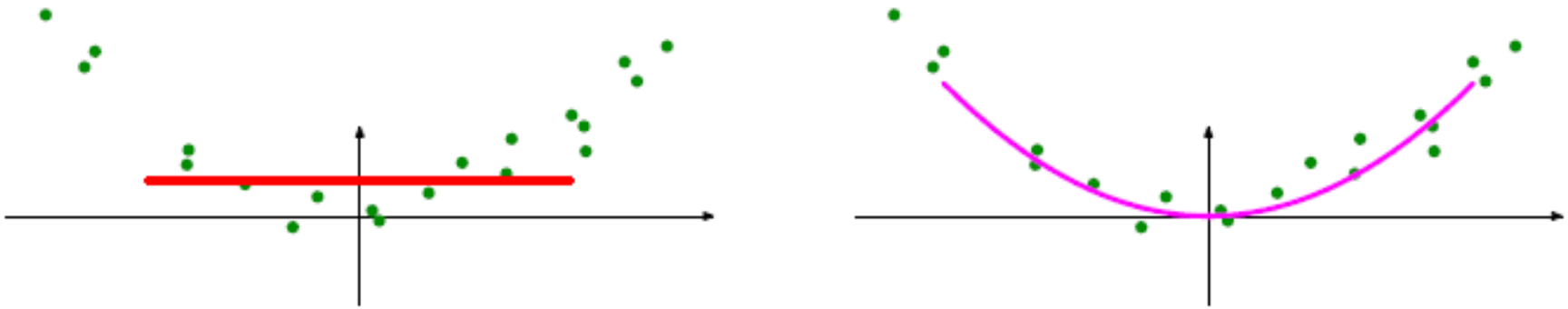
where $\mathbf{z} \in \mathbf{R}^k$

PCA failure



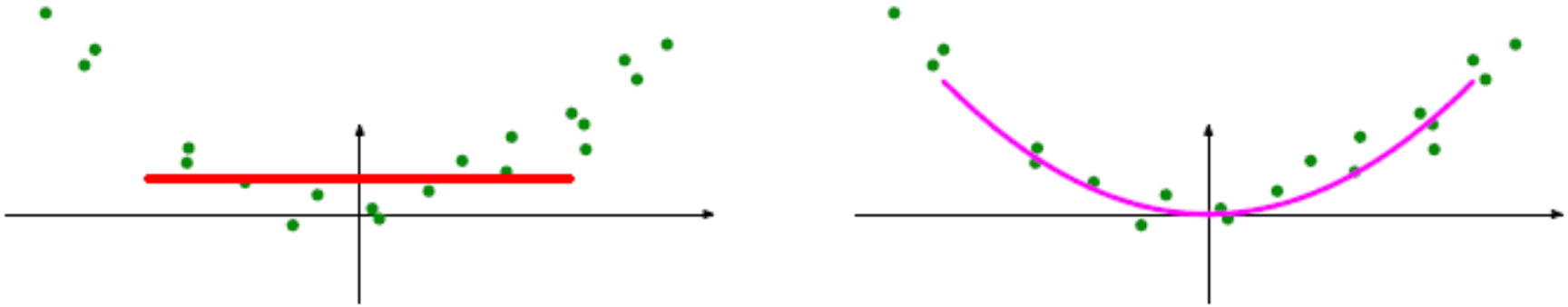
- In this example, we would really like the learned subspace to depend on x_1^2

PCA failure



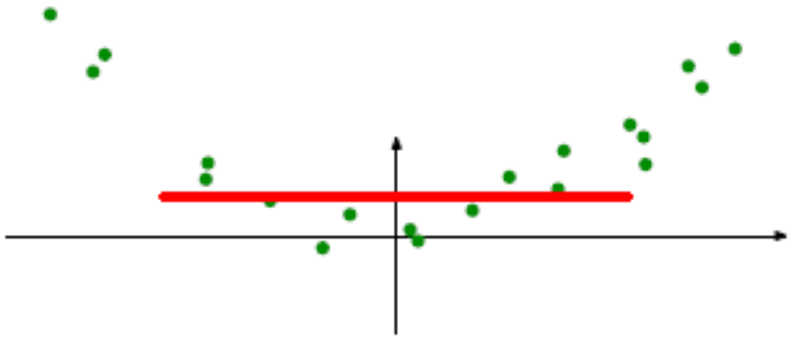
- In this example, we would really like the learned subspace to depend on x_1^2
- We could expand our input space to some other feature space, say $\phi(\mathbf{x}) = [x_1 \; x_1^2 \; x_2]^T$
- That would work!

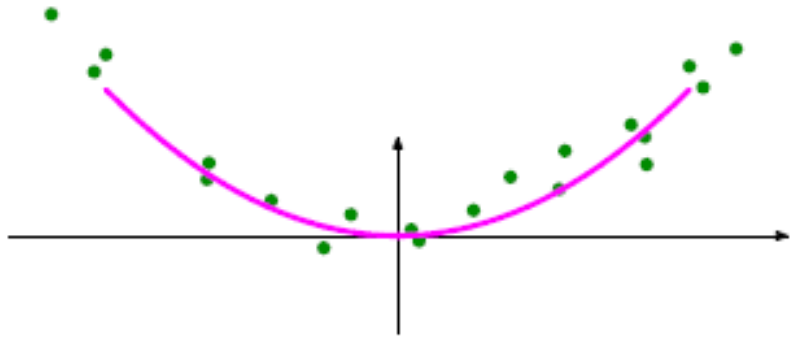
PCA failure



- But we don't *know* what the useful input features will be in any given data set

PCA failure





- But we don't *know* what the useful input features will be in any given data set
- We could expand our input space to an even bigger feature space... say
  $\phi(\mathbf{x}) = [x_1\ x_2\ x_1^2\ x_2^2\ x_1 x_2\ \sin(x_1) \ldots]^T$
- Well this is tedious... and we could need a very large $\phi(\mathbf{x})$

Toward kernel PCA

Let's use the intuition of multidimensional scaling

- PCA is equivalent to MDS $\Longrightarrow$ only needs the matrix of inner products $\mathbf{X}^T\mathbf{X}$
- Suppose we expand our input space to $\phi(\mathbf{x}) \Longrightarrow$ data matrix becomes $\Phi^T\Phi$
- We could do MDS on $\Phi^T\Phi$, but $\phi(\mathbf{x})$ might be very high-dimensional
- If only we had an easier way to compute the dot products $\phi(\mathbf{x}_1) \cdot \phi(\mathbf{x}_2)$...

Kernel PCA

Kernels provide an easier way to compute dot products in very high-dimensional feature spaces

- If $\phi(\cdot)$ lives in a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$[2] with kernel function $k(\cdot, \cdot)$, then we can avoid using $\phi(\cdot)$ altogether
- Do MDS on the inner products given by the kernel: $\phi(\mathbf{x}_1) \cdot \phi(\mathbf{x}_2) = k(\mathbf{x}_1, \mathbf{x}_2)$
- That is, seek eigenvectors of $\mathbf{K} = \Phi\Phi^T$, $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$
- This is kernel PCA
- So, kernel PCA is a nonlinear extension of MDS, where dot products are replaced with dot products in feature space

[2] This is circular, but let's say that means a space where there is a kernel that computes dot products between elements in the space

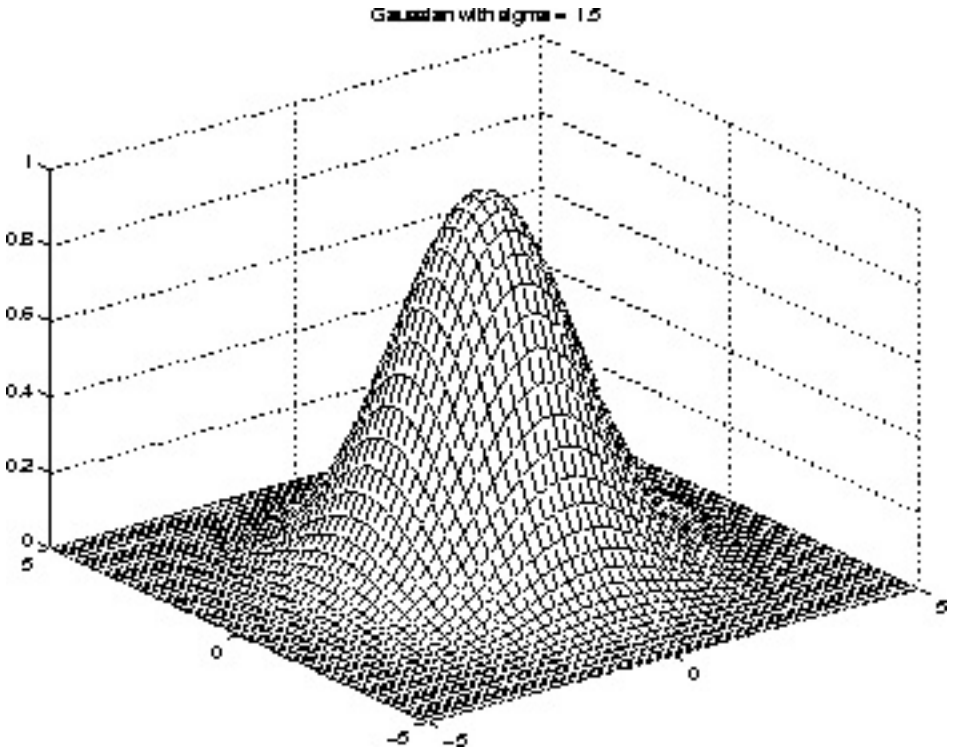
Kernel PCA: Choice of kernel

- In practice, we don't engineer $\phi(\cdot)$
- We directly think about $k(\cdot, \cdot)$

Kernel PCA: Choice of kernel

- In practice, we don't engineer $\phi(\cdot)$
- We directly think about $k(\cdot, \cdot)$
- One common choice: radial basis function (RBF), or Gaussian, kernel

  $k(\mathbf{x}_1, \mathbf{x}_2) = e^{\frac{-(\mathbf{x}_1 - \mathbf{x}_2)^2}{2\sigma^2}}$
- Corresponds to an implicit feature space of infinite dimensionality
- Has one tuning parameter, the standard deviation (radius) σ



Kernel PCA: Choice of kernel

- Another common choice: polynomial kernel
- $k(\mathbf{x}_1, \mathbf{x}_2) = (1 + \mathbf{x}_1^T \mathbf{x}_2)^p$
- Has one tuning parameter, the polynomial degree p
- $p = 2$ is quite common

Kernel PCA: Choice of kernel

- Another common choice: polynomial kernel
- $k(\mathbf{x}_1, \mathbf{x}_2) = (1 + \mathbf{x}_1^T \mathbf{x}_2)^p$
- Has one tuning parameter, the polynomial degree p
- $p = 2$ is quite common

In practice... just try a couple.

Kernel PCA: Choice of kernel

- Another common choice: polynomial kernel
- $k(\mathbf{x}_1, \mathbf{x}_2) = (1 + \mathbf{x}_1^T \mathbf{x}_2)^p$
- Has one tuning parameter, the polynomial degree p
- $p = 2$ is quite common

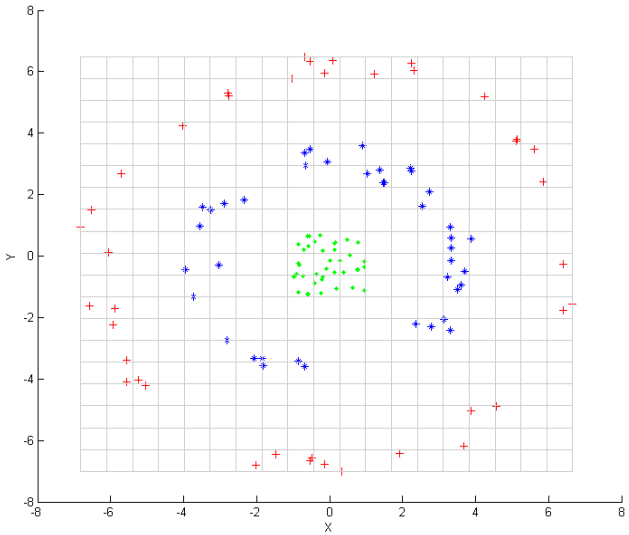
In practice... just try a couple. Or just one (RBF).

Kernel PCA: Mean normalization

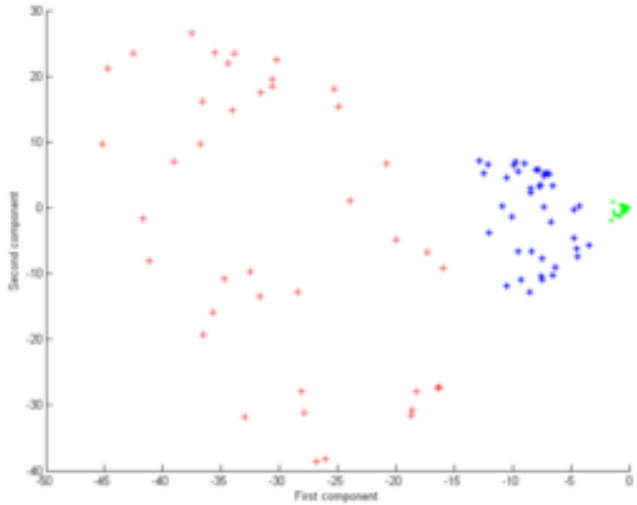
- We've implicitly assumed that the $\phi(\mathbf{x})$s are zero-mean
- If the inputs are not zero-mean, solve a slightly different eigenvalue problem, replacing $\mathbf{K}$ with $(\mathbf{I} - \mathbf{u}\mathbf{u}^T)\mathbf{K}(\mathbf{I} - \mathbf{u}\mathbf{u}^T)$, where $\mathbf{u} = \frac{1}{\sqrt{N}}[1 \dots 1]^T$
- (This can be easily derived by substituting the centered ϕs in the kernel matrix definition; see Bishop 12.3)

Kernel PCA: Example ☺

Input:



Output:



Kernel PCA: Example ☺

Original data



Data corrupted with Gaussian noise



Result after linear PCA



Result after kernel PCA, Gaussian kernel

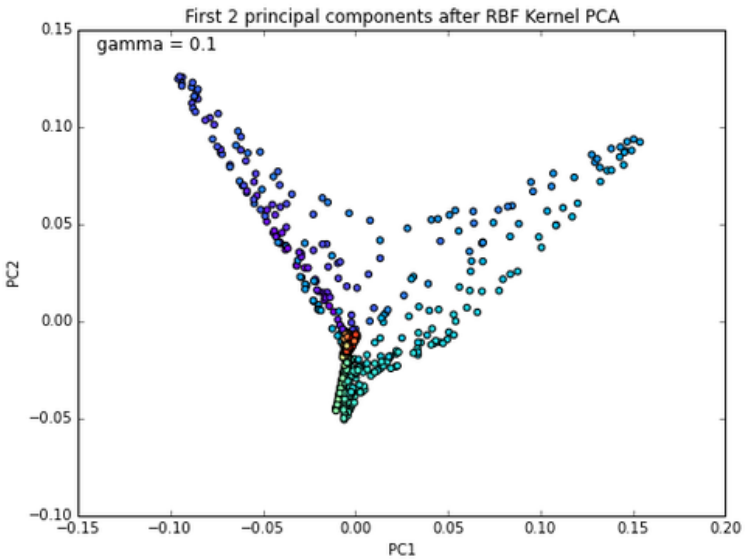


Kernel PCA: Example ☹

Input:



Output:



Kernel PCA: Connection with graph-based methods

KPCA looks very similar to some graph-based dimensionality reduction methods...

- Both are motivated as extensions of MDS
- Both involve finding eigenvectors of a matrix of inner products, or something like it
- E.g., in Isomap we do precisely that

So how does it differ?

Kernel PCA: Connection with graph-based methods

KPCA looks very similar to some graph-based dimensionality reduction methods...

- Both are motivated as extensions of MDS
- Both involve finding eigenvectors of a matrix of inner products, or something like it
- E.g., in Isomap we do precisely that

So how does it differ?

- In KPCA the inner product function (kernel) is fixed over the entire input space
- In graph-based methods the "kernel" is data-dependent

Out-of-sample extensions

- Most methods we have discussed so far today embed a specific data set in a lower-dimensionality space, i.e. map $\mathbf{x}$s to $\mathbf{z}$s
- How do we compute the projection of a new data point $\mathbf{x}_{\text{new}}$?

An alternative derivation of kernel PCA

Let's go back to the PCA formulation...

Covariance matrix in the feature space (assuming zero mean):

$$\mathbf{C} = \frac{1}{N} \sum_{i=1}^{N} \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T$$

The PCA projection vectors $\mathbf{w}$ are the eigenvectors of $\mathbf{C}$:

$$\mathbf{C}\mathbf{w} = \lambda \mathbf{w}$$

But we can't (or at least don't want to) write down $\phi(\mathbf{x})$!

An alternative derivation of kernel PCA

- Now we show that the eigenvectors can be expressed as linear combinations of the training set features, $\mathbf{w} = \sum_{i=1}^{N} \alpha_i \phi(\mathbf{x}_i)$:

$$\begin{aligned} \mathbf{C}\mathbf{w} &= \frac{1}{N} \sum_{i=1}^{N} \phi(\mathbf{x}_i)\phi(\mathbf{x}_i)^T \mathbf{w} = \lambda \mathbf{w} \\ \implies \mathbf{w} &= \frac{1}{\lambda N} \sum_{i=1}^{N} (\phi(\mathbf{x}_i) \cdot \mathbf{w})\phi(\mathbf{x}_i) \end{aligned}$$

- (This last equality requires a bit of algebra to show)

An alternative derivation of kernel PCA

- Now we show that the eigenvectors can be expressed as linear combinations of the training set features, $\mathbf{w} = \sum_{i=1}^{N} \alpha_i \phi(\mathbf{x}_i)$:

$$\begin{aligned} \mathbf{C}\mathbf{w} &= \frac{1}{N} \sum_{i=1}^{N} \phi(\mathbf{x}_i)\phi(\mathbf{x}_i)^T \mathbf{w} = \lambda \mathbf{w} \\ \implies \mathbf{w} &= \frac{1}{\lambda N} \sum_{i=1}^{N} (\phi(\mathbf{x}_i) \cdot \mathbf{w})\phi(\mathbf{x}_i) \end{aligned}$$

- (This last equality requires a bit of algebra to show)
- Now $(\phi(\mathbf{x}_i) \cdot \mathbf{w})$ is just some scalar, so this means $\mathbf{w}$ lies in the span of the $\phi(\mathbf{x}_i)$
- Combining all of the scalars, we have $\mathbf{w} = \sum_{i=1}^{N} \alpha_i \phi(\mathbf{x}_i)$

An alternative derivation of kernel PCA

Substituting the new expression for $\mathbf{w}$ into the eigenproblem (where we are indexing the eigenvectors/eigenvalues by j):

$$\begin{aligned}
\mathbf{C}\mathbf{w} = \frac{1}{N}\sum_{i=1}^{N}\phi(\mathbf{x}_i)\phi(\mathbf{x}_i)^T\left(\sum_{l=1}^{N}\alpha_{jl}\phi(\mathbf{x}_l)\right) &= \lambda_j\sum_{l=1}^{N}\alpha_{jl}\phi(\mathbf{x}_l) \\
\implies \frac{1}{N}\sum_{i=1}^{N}\phi(\mathbf{x}_i)\left(\sum_{l=1}^{N}\alpha_{jl}k(\mathbf{x}_i,\mathbf{x}_l)\right) &= \lambda_j\sum_{l=1}^{N}\alpha_{jl}\phi(\mathbf{x}_l)
\end{aligned}$$

After a bit more massaging, this becomes

$$\mathbf{K}\alpha_j = N\lambda_j\alpha_j$$

Alternative derivation of kernel PCA

Finally, the eigenvector normalization constraint produces a constraint on α_j:

$$\begin{aligned} \mathbf{w}_j^T \mathbf{w}_j = 1 \quad &\Longrightarrow \quad \sum_{k=1}^{N} \sum_{l=1}^{N} \alpha_{jl} \alpha_{jk} \phi(\mathbf{x}_l)^T \phi(\mathbf{x}_k) = 1 \\ &\Longrightarrow \quad \alpha_j^T \mathbf{K} \alpha_j = 1 \end{aligned}$$

Now to project a new point $\mathbf{x}$, we project its feature vector onto each eigenvector $\mathbf{w}_j$:

$$\begin{aligned} \mathbf{z}_j &= \phi(\mathbf{x})^T \mathbf{w}_j \\ &= \sum_{i=1}^{N} \alpha_{ji} \phi(\mathbf{x})^T \phi(\mathbf{x}_i) = \sum_{i=1}^{N} \alpha_{ji} k(\mathbf{x}, \mathbf{x}_i) \end{aligned}$$

This allows us to solve the kernel version of the formulation (the "dual") and then project a new example not in the training set $\Longrightarrow$ an out-of-sample extension

Kernel PCA: summary

- Choose kernel $k(\cdot, \cdot)$
- Form the $N \times N$ (normalized) kernel matrix $\mathbf{K}$
- Solve the eigenproblem $\mathbf{K}\alpha_j = \lambda_j \alpha_j$
- Given any data point, its k-dimensional projection will be

$$\mathbf{z}_j = \sum_{i=1}^{N} \alpha_{ji} k(\mathbf{x}, \mathbf{x}_i), \quad j = 1, \ldots, k$$

Scaling kernel methods

What if the kernel matrix is too large to work with? (In practice, this happens all the time... > 10K examples might be too large for this purpose!)

Scaling kernel methods

What if the kernel matrix is too large to work with? (In practice, this happens all the time... > 10K examples might be too large for this purpose!)

- Apply the kernel method on a manageable-sized subset of the entire data set
- Use kernel approximation techniques
- Don't use kernel methods (use neural networks, probably) – next time!

Scaling kernel methods: Approximation

Approximate the kernel matrix with a low-rank approximation: $\mathbf{K} \approx \mathbf{F}^T\mathbf{F}$, where $\mathbf{F} \in \mathbf{R}^{M \times N}$

- Typically $k << M << N$
- This is equivalent to representing the input in the high-dimensional feature space implied by the kernel, then projecting that down to M dimensions
- Then do the dimensionality reduction from $\mathbf{R}^M$ to $\mathbf{R}^k$
- Is this begging the question?

Scaling kernel methods: Approximation

- There are a few techniques that won't break the computation bank...
- One remarkably simple technique for RBF kernels: random Fourier features (Rahimi & Recht 2008)
- Replace each data point $\mathbf{x}_i$ with

$$[\cos(w_1^T \mathbf{x}_i + b_1) \dots \cos(w_M^T \mathbf{x}_i + b_M)]^T = F_i$$

where

$$\begin{aligned} b_1, \dots, b_M &\sim \text{Unif}[0, 2\pi] \\ w_1, \dots, w_M &\sim \mathcal{N}\left(0, \frac{2}{\sigma^2} I\right) \end{aligned}$$

- Turns out to be a good M-dimensional approximation to the kernel's induced feature space (due to Fourier transform properties)
- What should M be? As large as you can handle